PROGRAM FLOWCHARTS :PARTS A TO J

FLOWCHART INDEX

PART A: ACCESS CONTROL

- login(username, password)
- get_permissions(role)
- show_manage_users()

PART B: WORKER HEALTH & SAFETY

- worker_risk(ppe, fatigue, near_misses, training, previous_incidents)
- safety_warning(ppe, training, fatigue, near_misses, previous_incidents)
- show_worker_safety()

PART C: SAFETY INCIDENT DATABASE

- show_safety_incidents()

PART D: EQUIPMENT DATABASE

- show_equipment_database()

PART E: EQUIPMENT CONDITION MONITORING

- equipment_condition(temperature, vibration)
- show_equipment_condition()

PART F: MAINTENANCE MONITORING

- maintenance_check(operating_hours, next_maintenance, current_status)
- show_maintenance()

PART G: RISK ASSESSMENT

- risk_assessment(likelihood, consequence)
- show_risk_assessment()

PART H: ALERTS & NOTIFICATIONS

- equipment_alert(temperature, vibration, brakes, tyres, engine, maintenance)

- show_alerts(role)

PART I: MINING MONITORING DASHBOARD

- show_dashboard()

PART J: INTERACTIVE DATA ANALYSIS

- most_common(column)
- highest_downtime()
- lowest_availability()
- highest_temperature()
- highest_vibration()
- serious_incident_count()
- ppe_compliance_percentage()
- answer_question(question)
- show_data_analysis(role)

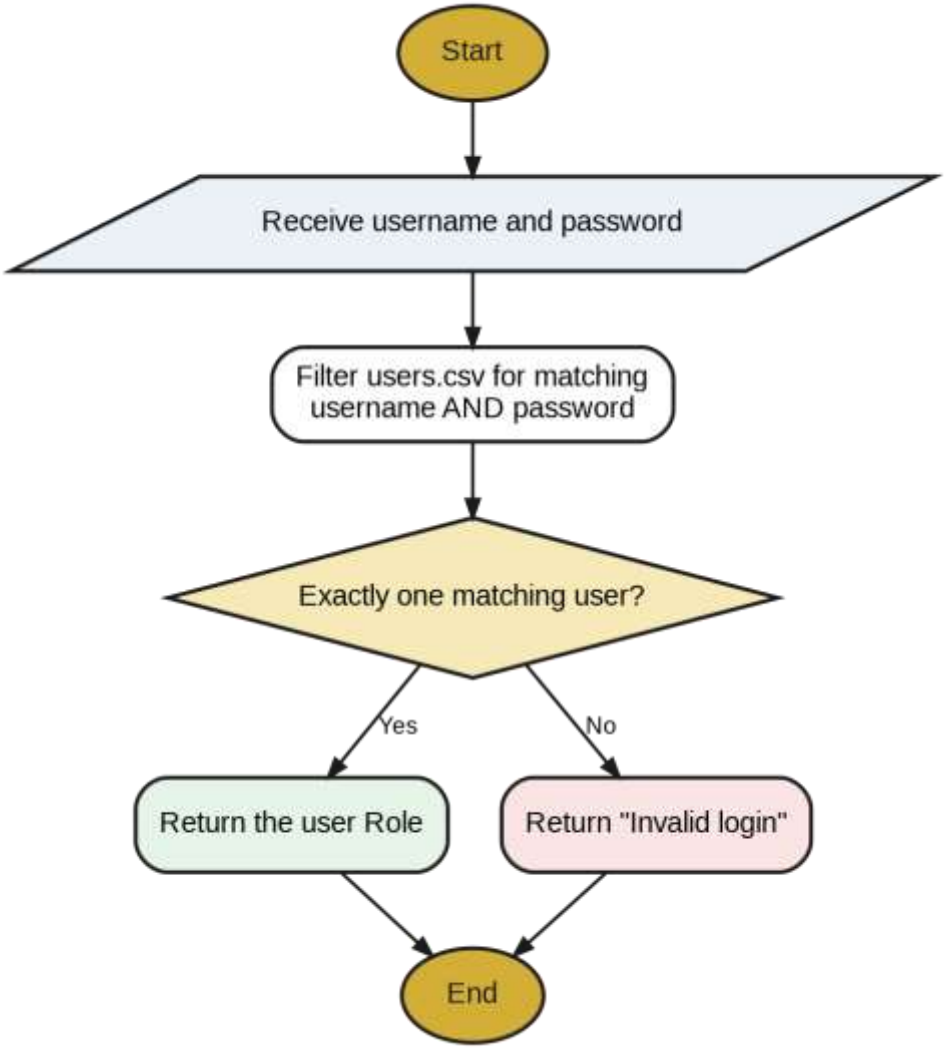
APPENDIX: REPORTS FUNCTION

- show_reports(role)

PART A: ACCESS CONTROL

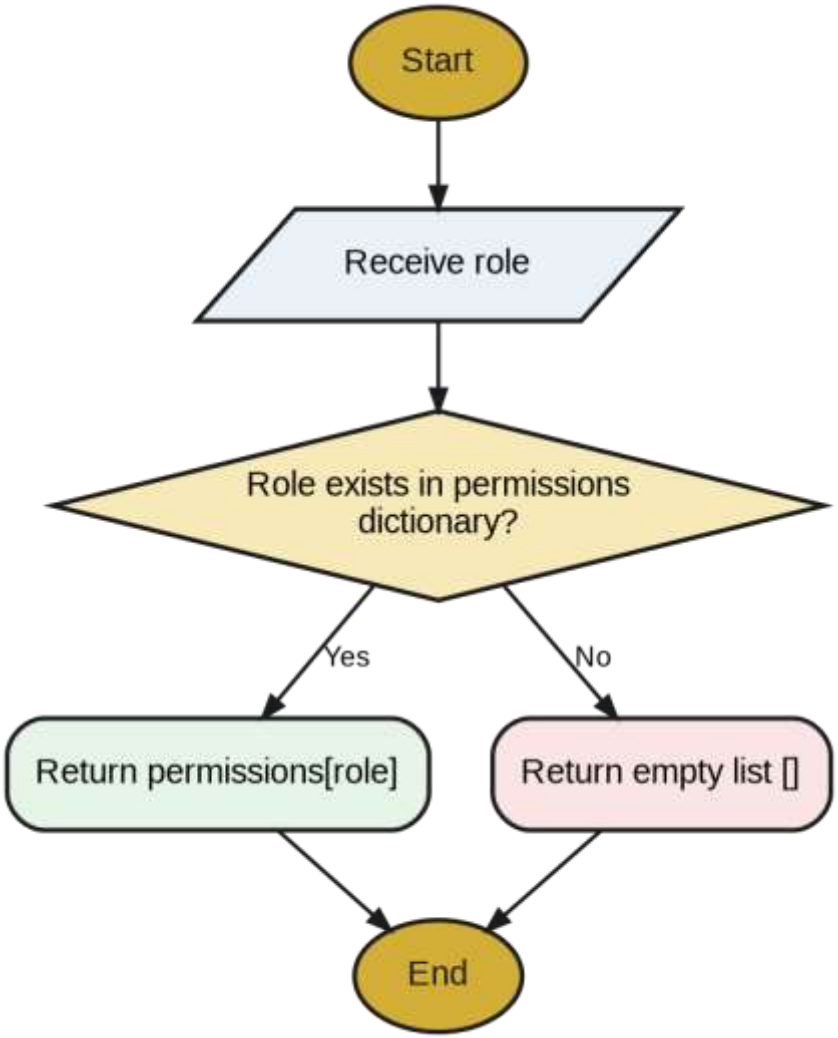
login(username, password)

Checks the entered credentials and returns the user role when the login is valid.



get_permissions(role)

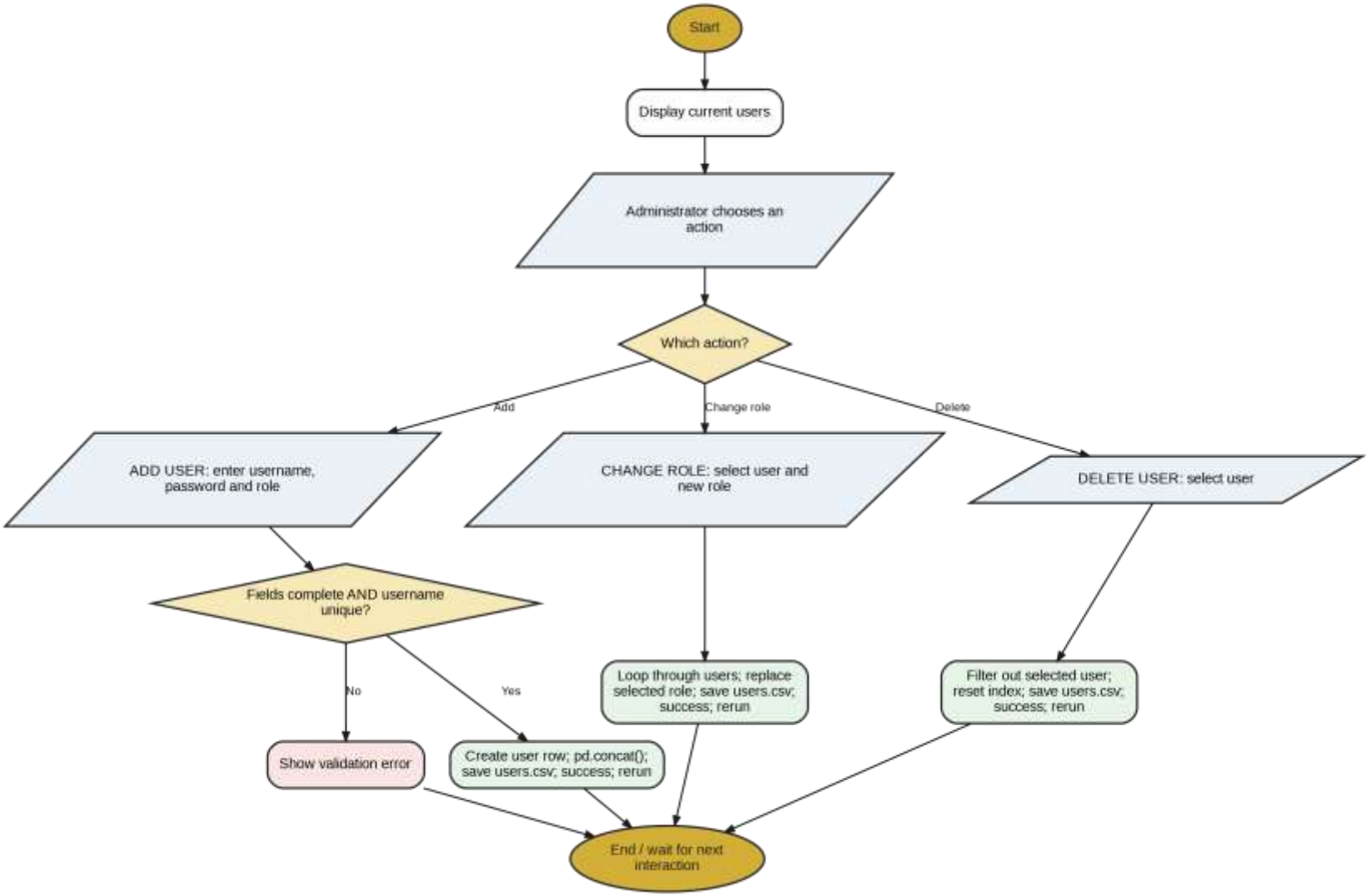
Returns the navigation pages allowed for the logged-in role.



PART A: ACCESS CONTROL

show_manage_users()

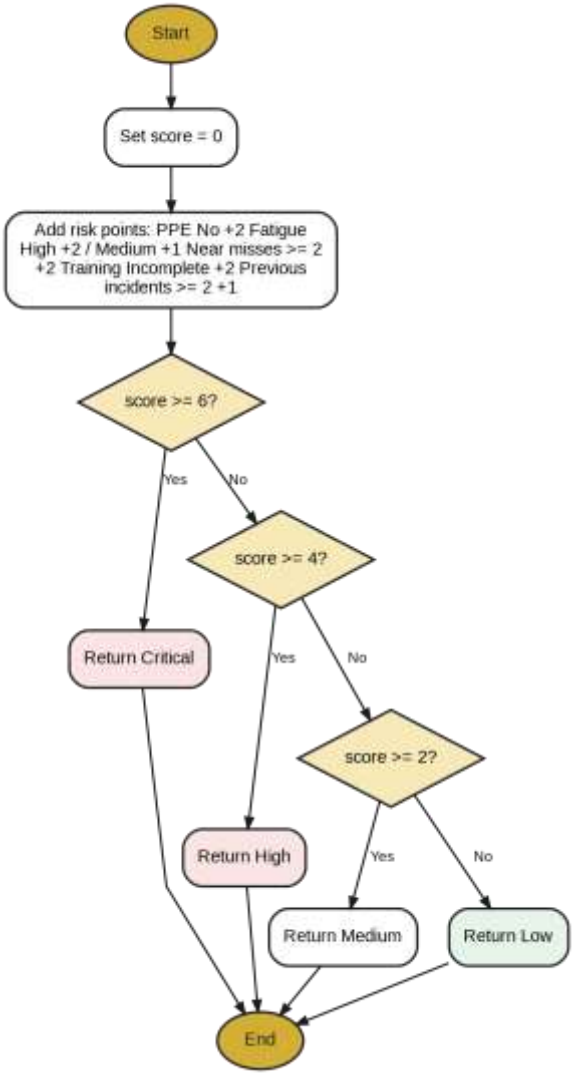
Lets the Administrator view users, add a user, change a role, or delete a user.



PART B: WORKER HEALTH & SAFETY

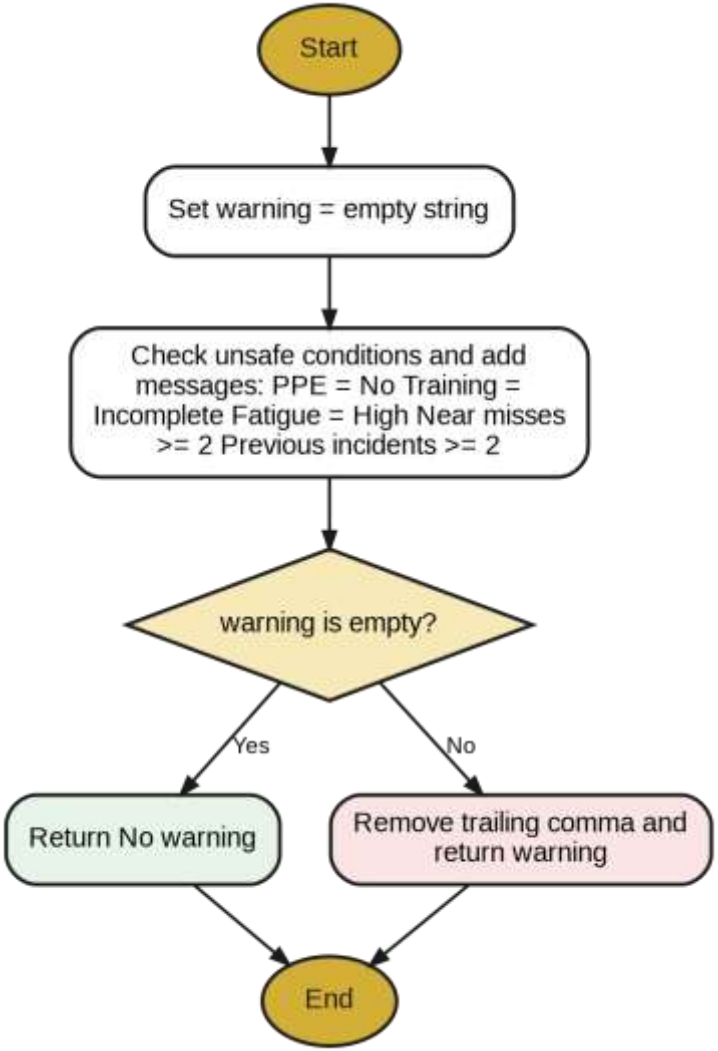
worker_risk(ppe, fatigue, near_misses, training, previous_incidents)

Calculates a worker risk score and classifies the worker as Low, Medium, High, or Critical risk.



safety_warning(ppe, training, fatigue, near_misses, previous_incidents)

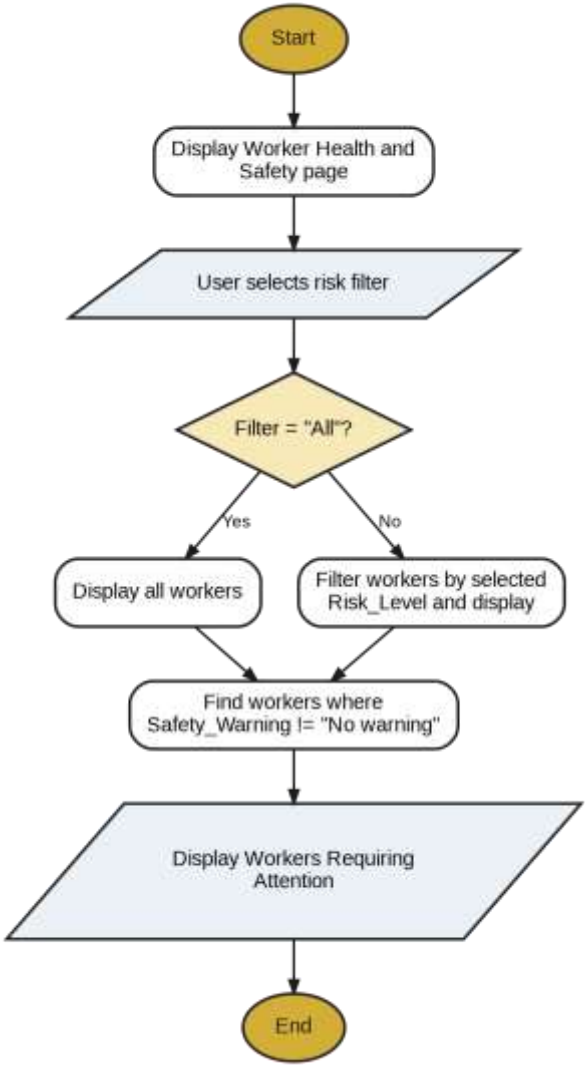
Builds a safety warning message from unsafe worker conditions.



PART B: WORKER HEALTH & SAFETY

show_worker_safety()

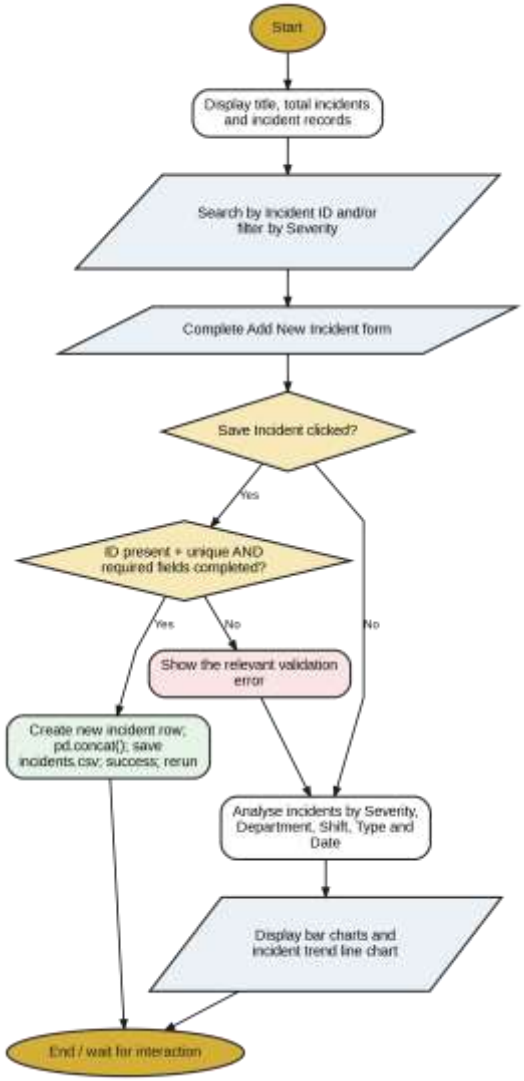
Displays worker safety data, filters by risk level, and shows workers who require attention.



PART C: SAFETY INCIDENT DATABASE

show_safety_incidents()

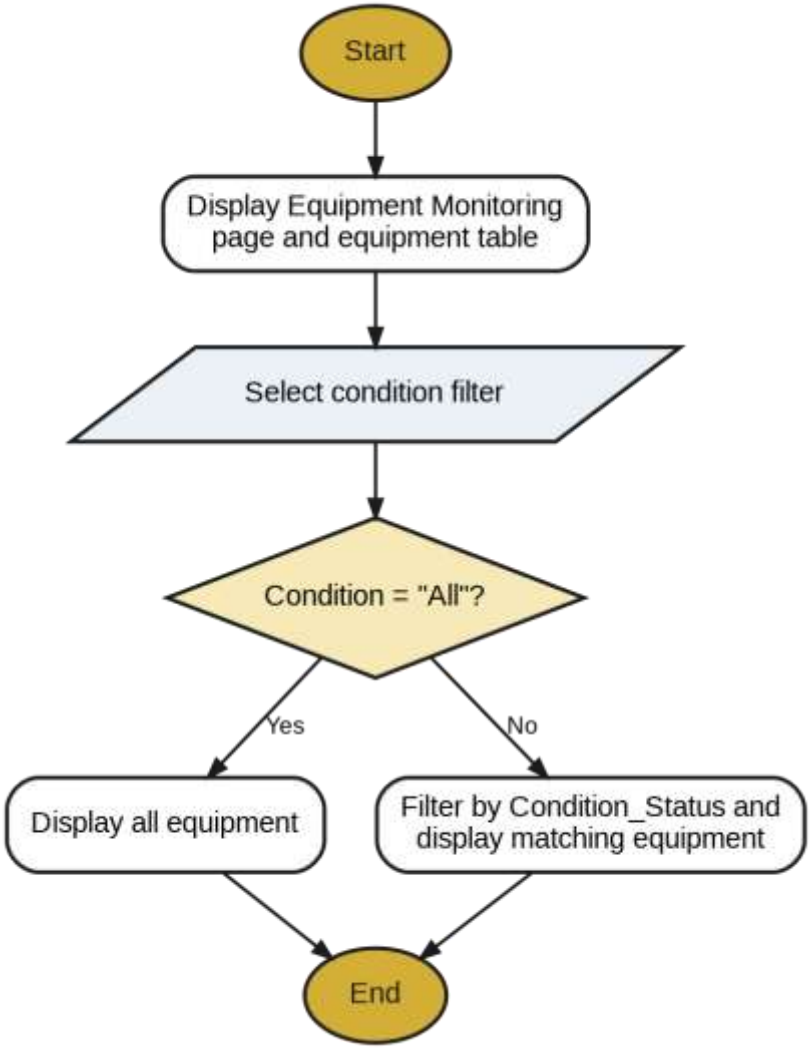
Displays, searches, filters, records, validates, saves, and analyses safety incidents.



PART D: EQUIPMENT DATABASE

show_equipment_database()

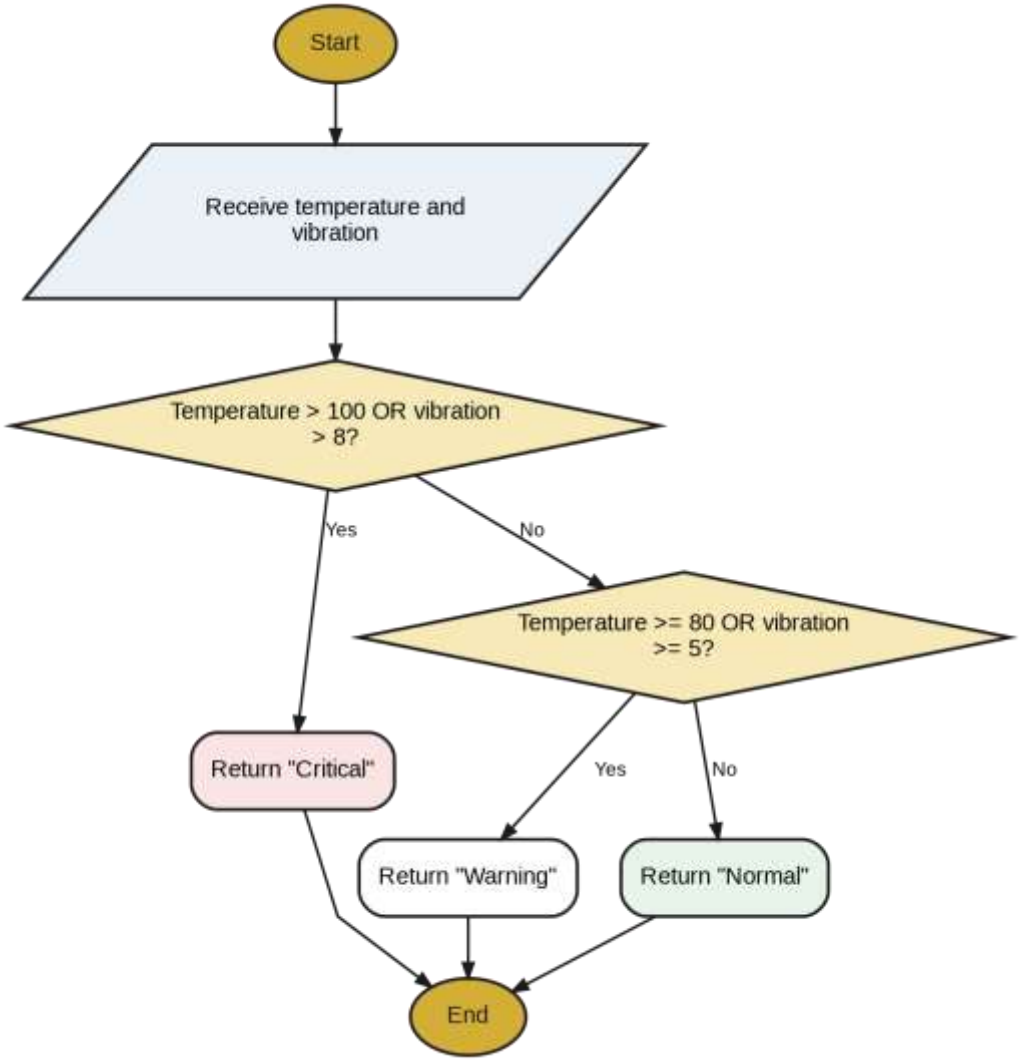
Displays the equipment database and lets the user filter equipment by condition status.



PART E: EQUIPMENT CONDITION MONITORING

equipment_condition(temperature, vibration)

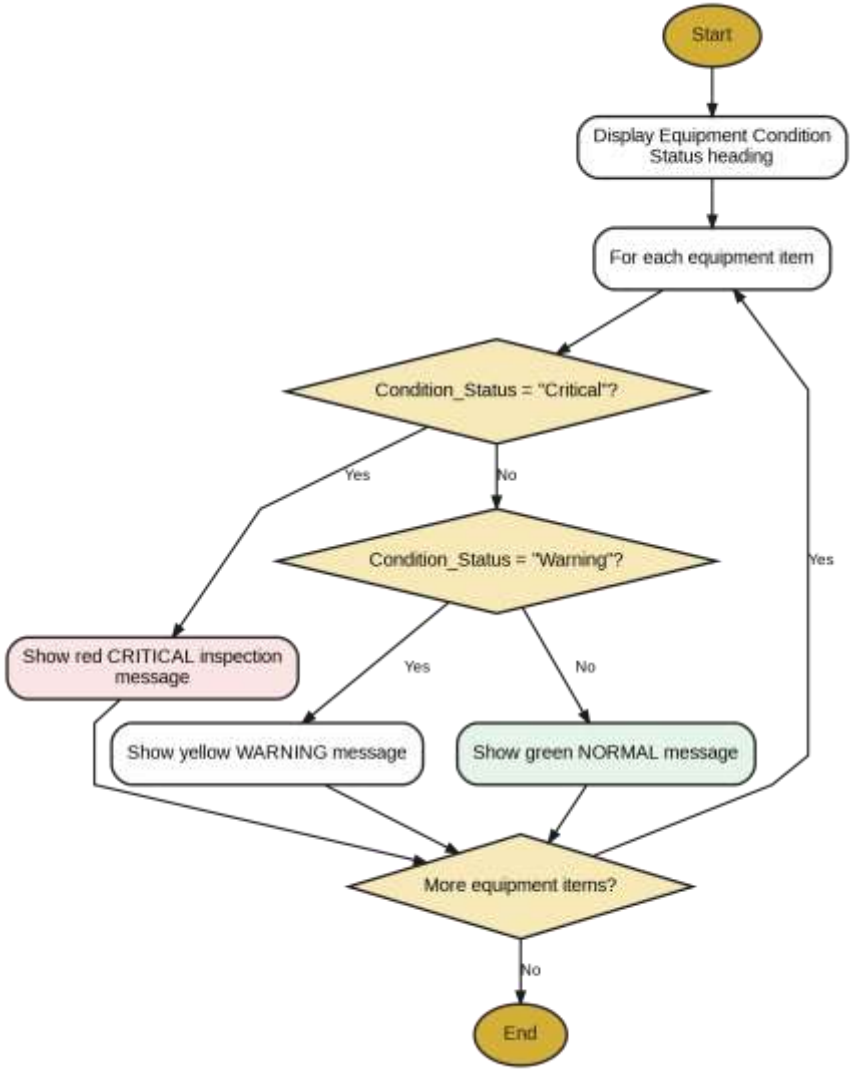
Classifies equipment condition using the project temperature and vibration thresholds.



PART E: EQUIPMENT CONDITION MONITORING

show_equipment_condition()

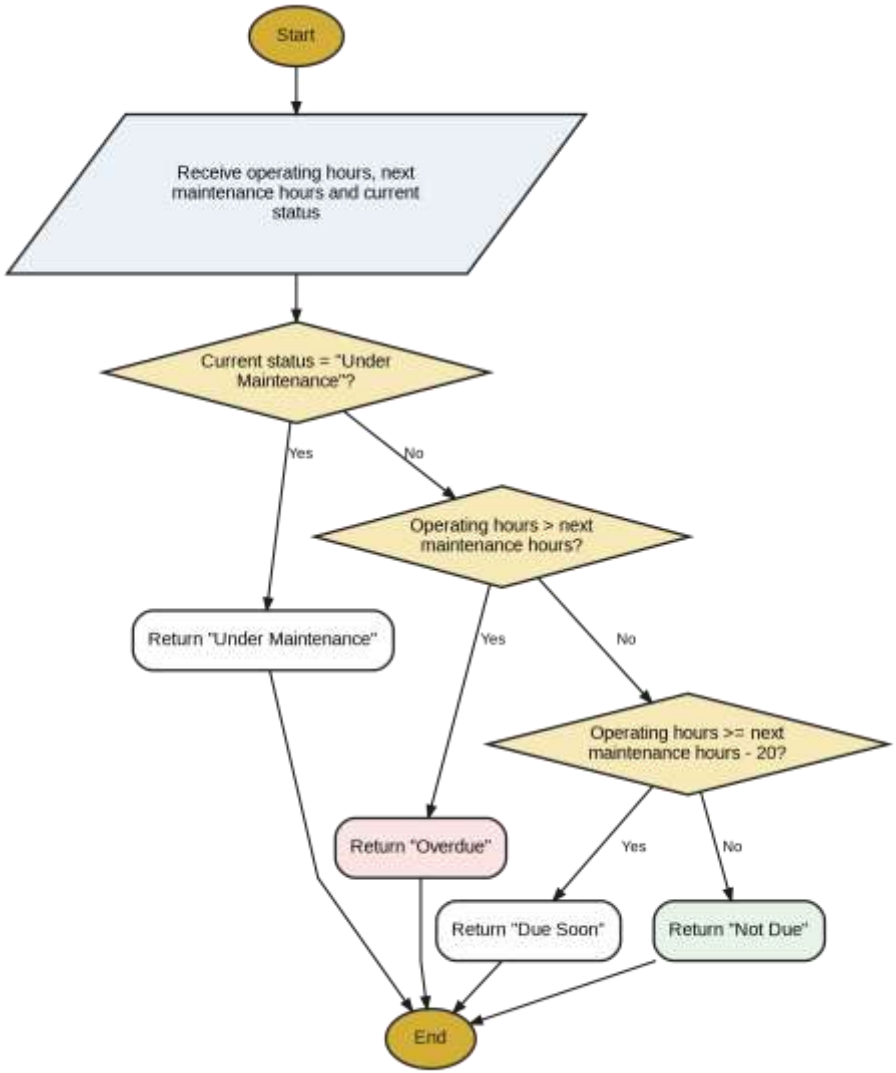
Shows a Normal, Warning, or Critical monitoring message for every equipment item.



PART F: MAINTENANCE MONITORING

maintenance_check(operating_hours, next_maintenance, current_status)

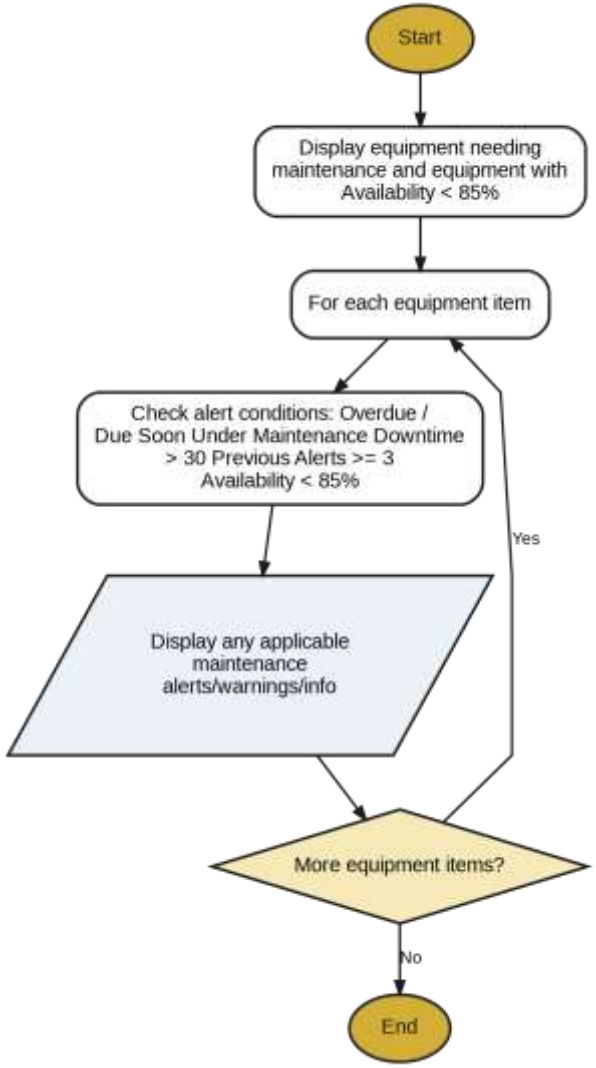
Determines whether equipment is under maintenance, overdue, due soon, or not due.



PART F: MAINTENANCE MONITORING

show_maintenance()

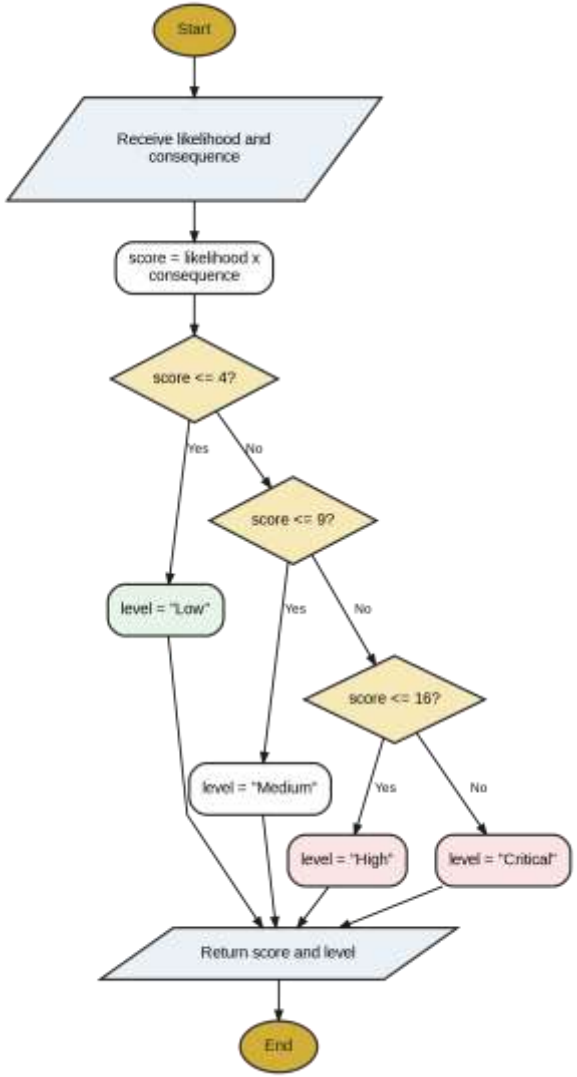
Displays maintenance attention items, low availability equipment, and maintenance alerts.



PART G: RISK ASSESSMENT

risk_assessment(likelihood, consequence)

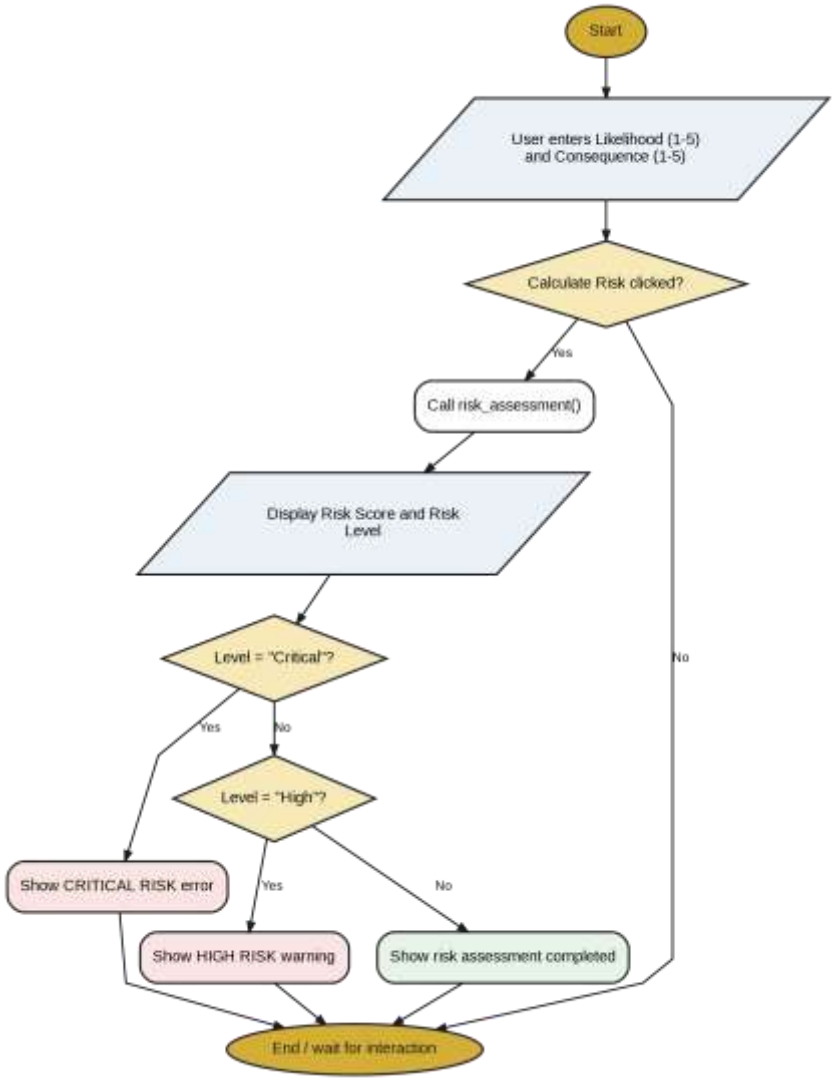
Calculates risk score as Likelihood x Consequence and assigns the project risk level.



PART G: RISK ASSESSMENT

show_risk_assessment()

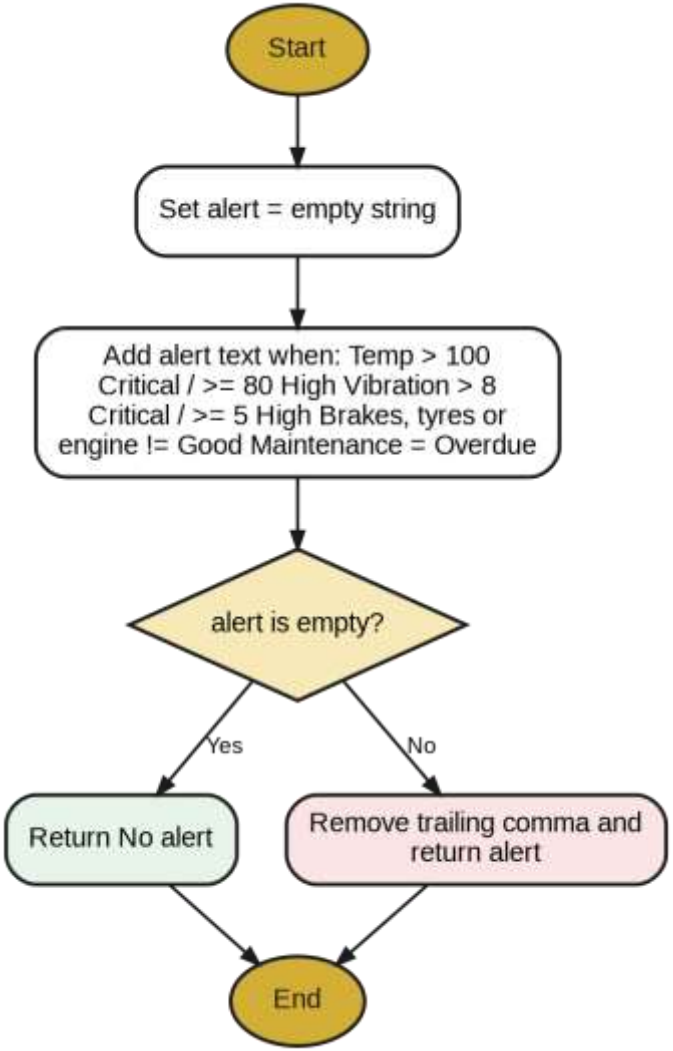
Collects likelihood and consequence from the user, calculates risk, and displays the correct risk message.



PART H: ALERTS & NOTIFICATIONS

equipment_alert(temperature, vibration, brakes, tyres, engine, maintenance)

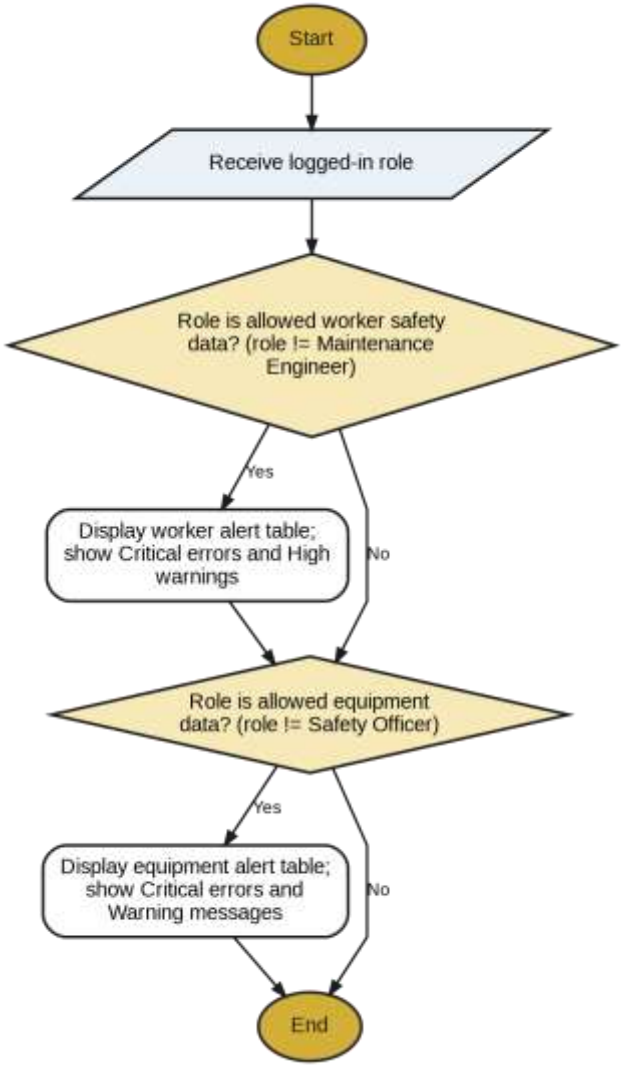
Builds an equipment alert message from temperature, vibration, component condition, and maintenance status.



PART H: ALERTS & NOTIFICATIONS

show_alerts(role)

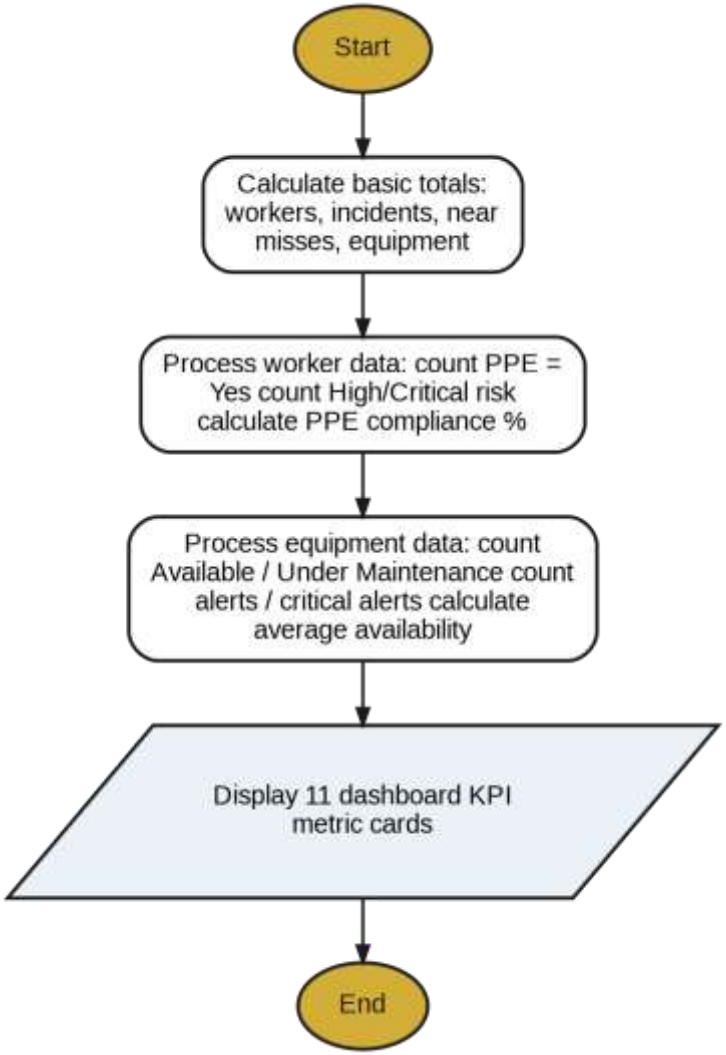
Shows worker and/or equipment alerts according to the logged-in user role.



PART I: MINING MONITORING DASHBOARD

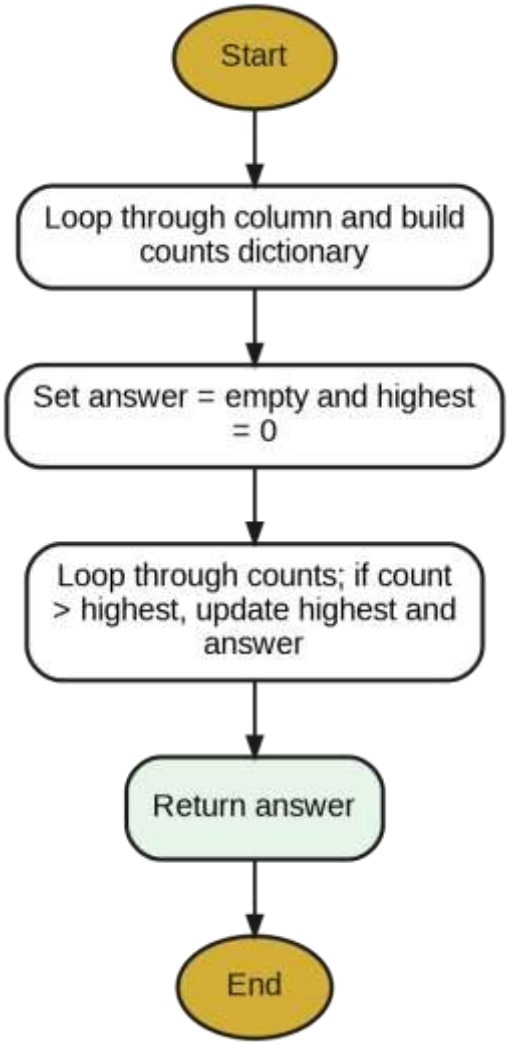
show_dashboard()

Calculates the eleven dashboard KPIs and displays them as metric cards.



most_common(column)

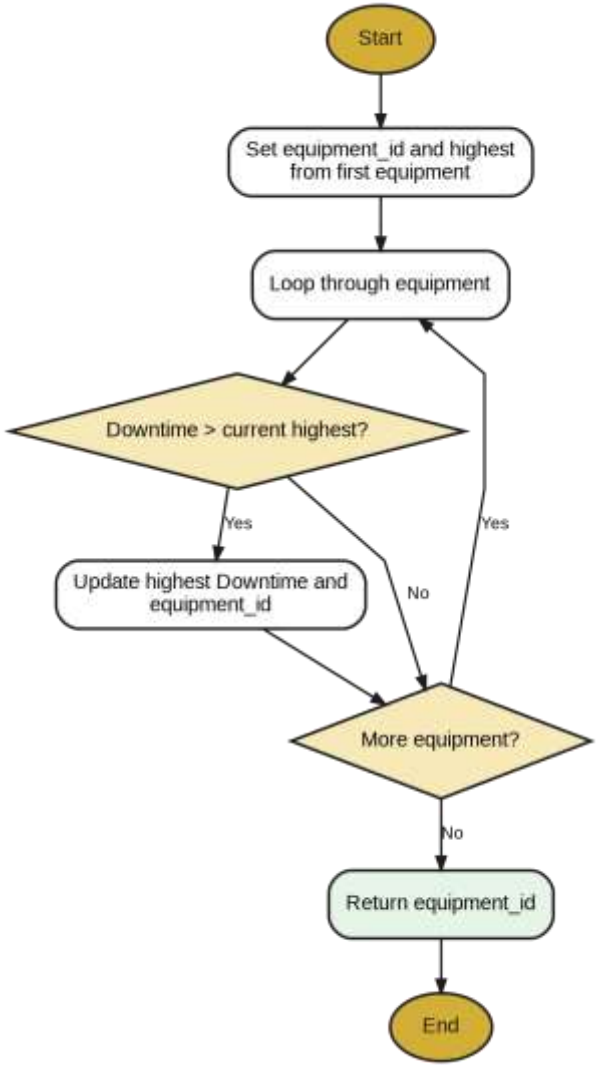
Counts values in a column and returns the value that occurs most often.



PART J: INTERACTIVE DATA ANALYSIS

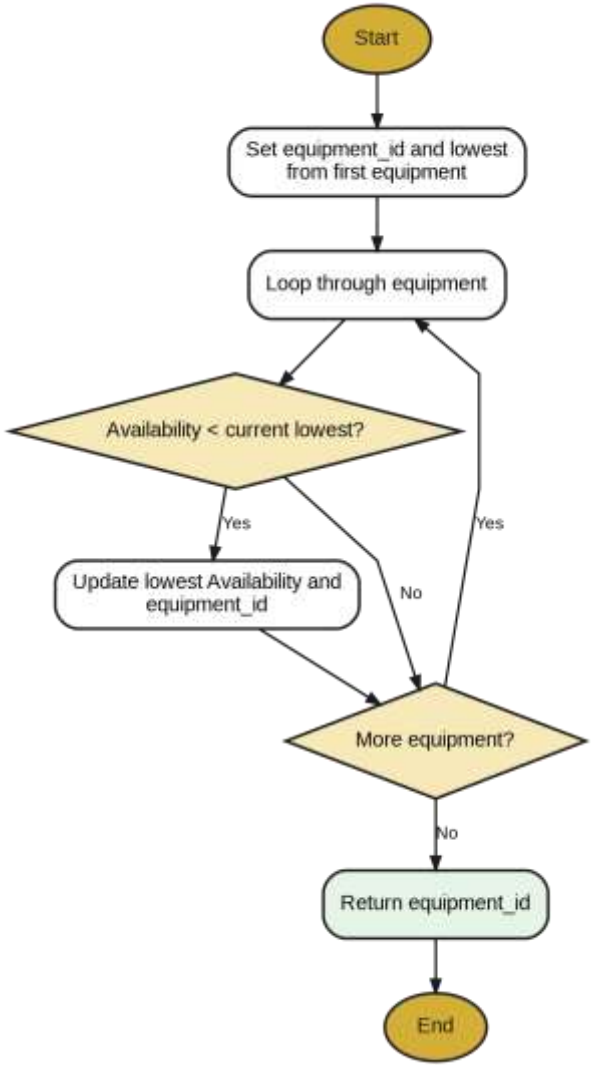
highest_downtime()

Returns the equipment with the highest downtime.



lowest_availability()

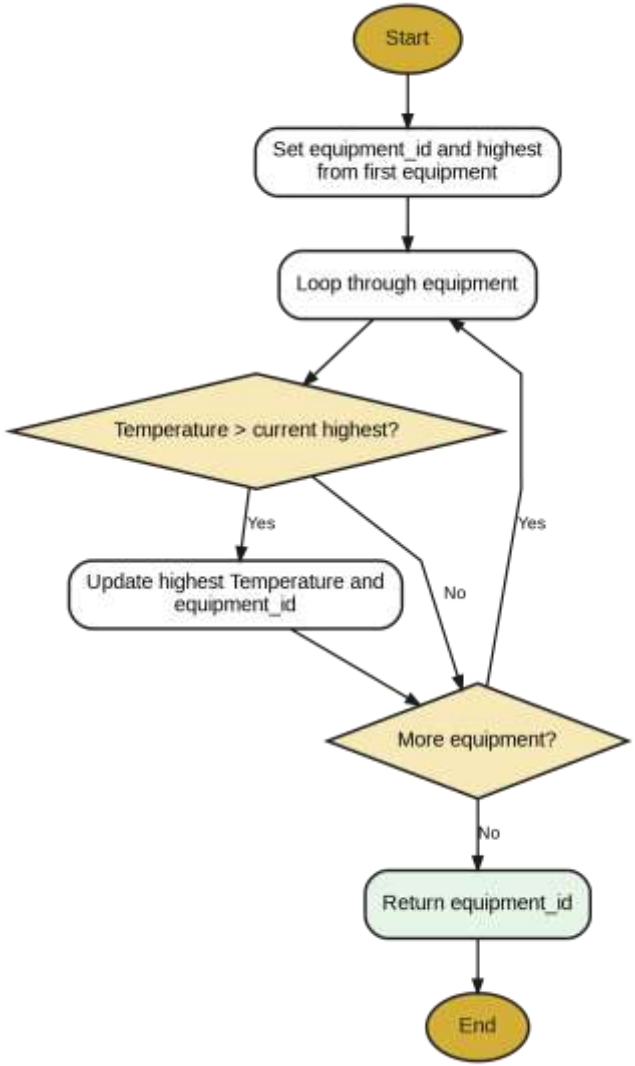
Returns the equipment with the lowest availability.



PART J: INTERACTIVE DATA ANALYSIS

highest_temperature()

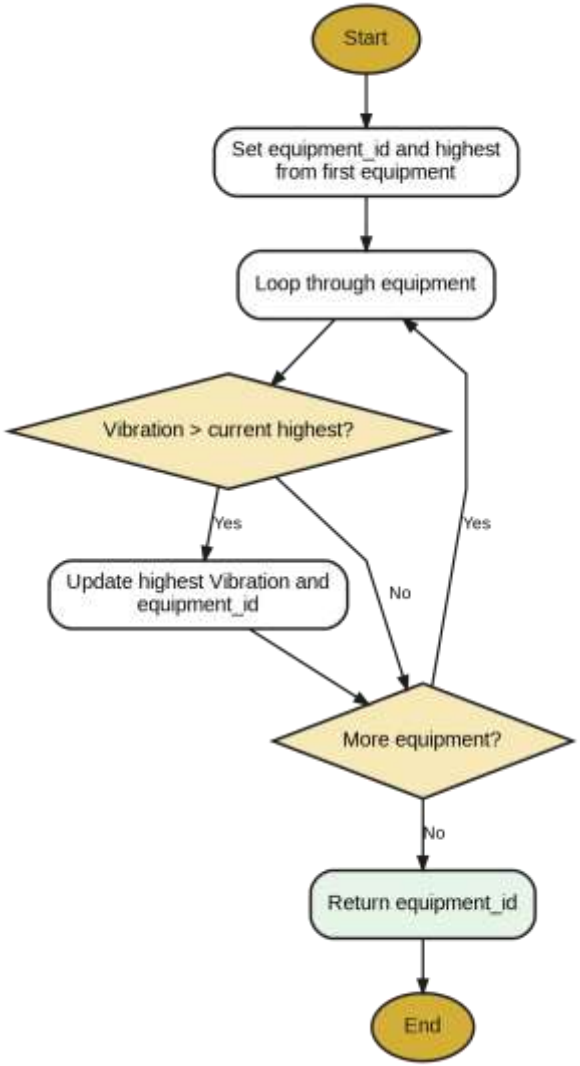
Returns the equipment with the highest temperature.



PART J: INTERACTIVE DATA ANALYSIS

highest_vibration()

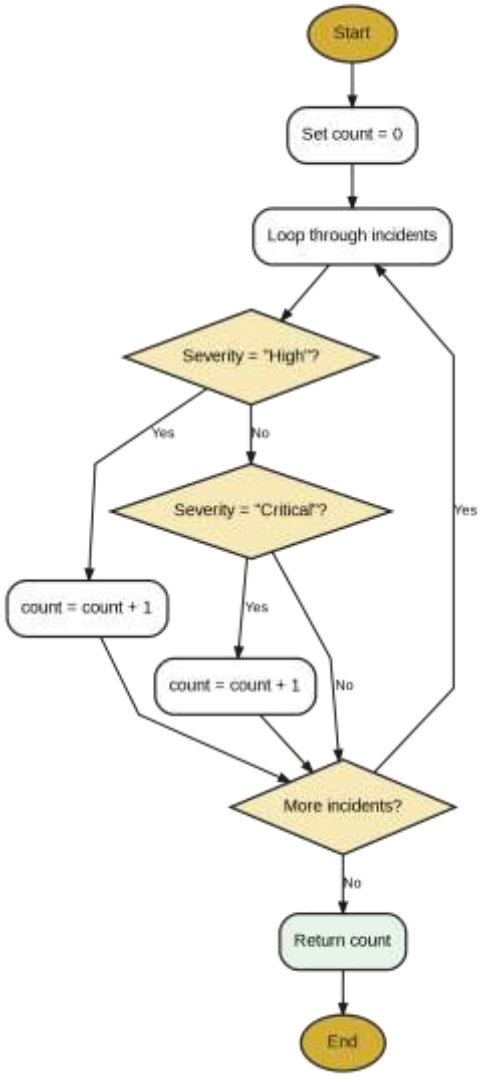
Returns the equipment with the highest vibration.



PART J: INTERACTIVE DATA ANALYSIS

serious_incident_count()

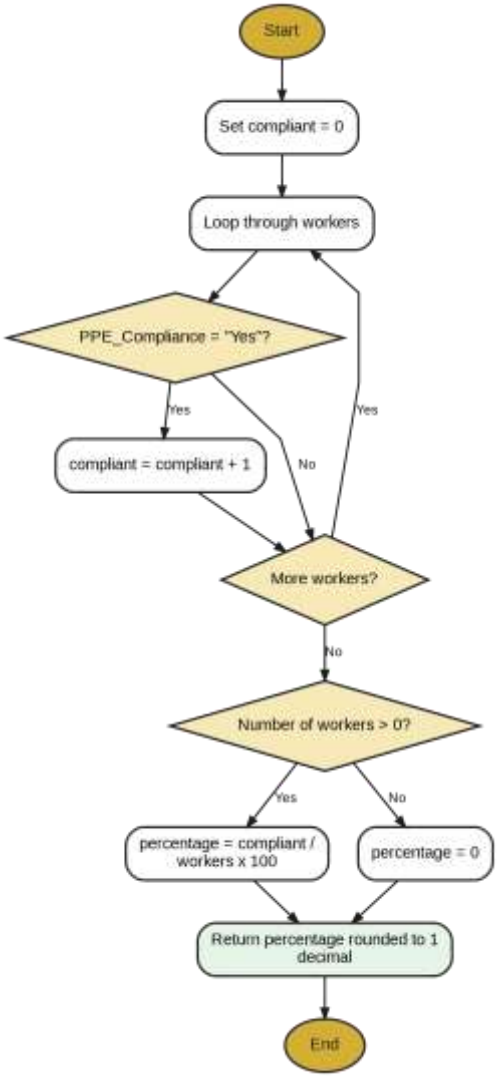
Counts incidents classified as High or Critical severity.



PART J: INTERACTIVE DATA ANALYSIS

ppe_compliance_percentage()

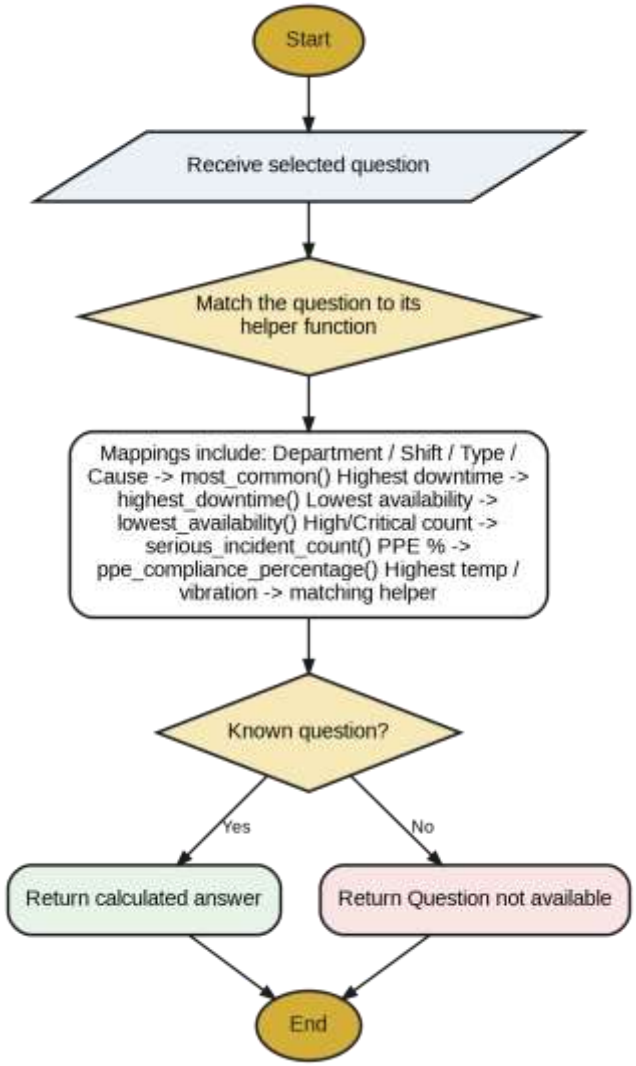
Calculates the percentage of workers whose PPE_Compliance is Yes.



PART J: INTERACTIVE DATA ANALYSIS

answer_question(question)

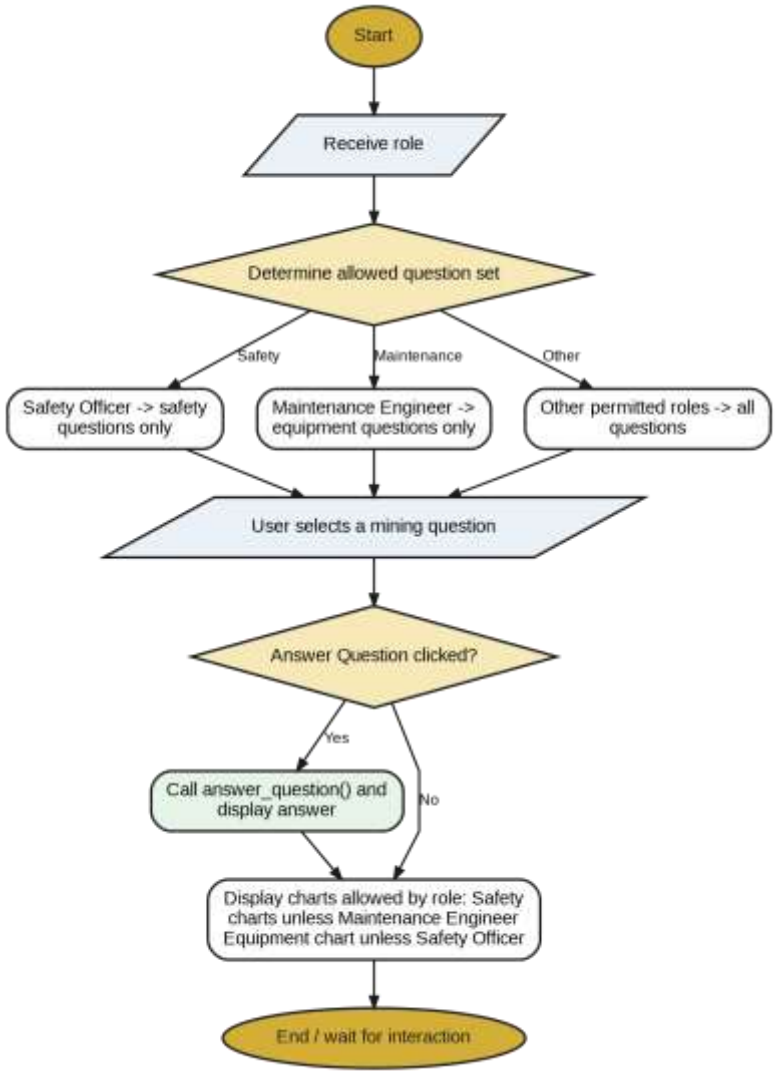
Routes the selected analysis question to the correct helper calculation and returns the answer.



PART J: INTERACTIVE DATA ANALYSIS

show_data_analysis(role)

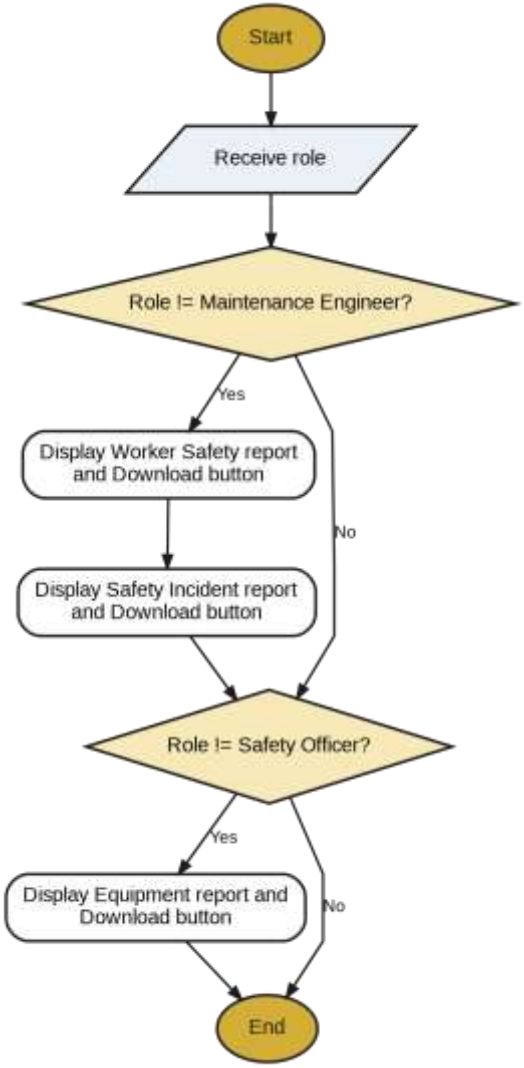
Restricts analysis questions and charts by role, then answers the selected mining question.



APPENDIX: REPORTS FUNCTION

show_reports(role)

Displays and downloads only the report data that the logged-in role is allowed to access.



PART A: ACCESS CONTROL

login(username, password)

Purpose: Checks the login details and returns the user role when the credentials are valid.

```
BEGIN login
    INPUT username, password
    FIND user in users data WHERE Username = username AND Password = password
    IF exactly one matching user is found THEN
        RETURN the user Role
    ELSE
        RETURN "Invalid login"
    ENDIF
END login
```

get_permissions(role)

Purpose: Returns the list of pages that the selected role is allowed to access.

```
BEGIN get_permissions
    INPUT role
    IF role exists in permissions dictionary THEN
        RETURN permissions for role
    ELSE
        RETURN empty list
    ENDIF
END get_permissions
```

show_manage_users()

Purpose: Allows the Administrator to view users, add users, change roles and delete users.

```
BEGIN show_manage_users
    DISPLAY current Username and Role columns

    INPUT new username, new password, new role
    IF Add User button is clicked THEN
        IF username OR password is blank THEN
            DISPLAY error message
        ELSE IF username already exists THEN
            DISPLAY duplicate username error
        ELSE
            CREATE new user record
            ADD new user record to users data
            SAVE users data to users.csv
            DISPLAY success message
            RERUN application
```

```
        ENDIF
    ENDIF

    INPUT selected user and selected new role
    IF Change Role button is clicked THEN
        FOR EACH user in users data
            IF Username = selected user THEN
                SET Role = selected new role
            ENDIF
        ENDFOR
        SAVE users data to users.csv
        DISPLAY success message
        RERUN application
    ENDIF

    INPUT user to delete
    IF Delete User button is clicked THEN
        REMOVE selected user from users data
        RESET row index
        SAVE users data to users.csv
        DISPLAY success message
        RERUN application
    ENDIF
END show_manage_users
```

PART B: WORKER HEALTH AND SAFETY MONITORING

worker_risk(ppe, fatigue, near_misses, training, previous_incidents)

Purpose: Calculates a worker risk level using PPE, fatigue, near misses, training and previous incidents.

```
BEGIN worker_risk
    INPUT ppe, fatigue, near_misses, training, previous_incidents
    SET score = 0

    IF ppe = "No" THEN
        score = score + 2
    ENDIF

    IF fatigue = "High" THEN
        score = score + 2
    ELSE IF fatigue = "Medium" THEN
        score = score + 1
    ENDIF

    IF near_misses >= 2 THEN
        score = score + 2
    ENDIF

    IF training = "Incomplete" THEN
        score = score + 2
    ENDIF

    IF previous_incidents >= 2 THEN
        score = score + 1
    ENDIF

    IF score >= 6 THEN
        RETURN "Critical"
    ELSE IF score >= 4 THEN
        RETURN "High"
    ELSE IF score >= 2 THEN
        RETURN "Medium"
    ELSE
        RETURN "Low"
    ENDIF
END worker_risk
```

safety_warning(ppe, training, fatigue, near_misses, previous_incidents)

Purpose: Builds a warning message describing unsafe worker conditions.

```
BEGIN safety_warning
    INPUT ppe, training, fatigue, near_misses, previous_incidents
    SET warning = empty text

    IF ppe = "No" THEN
        ADD "PPE non-compliance" to warning
    ENDIF
    IF training = "Incomplete" THEN
        ADD "Training incomplete" to warning
    ENDIF
    IF fatigue = "High" THEN
        ADD "High fatigue" to warning
    ENDIF
    IF near_misses >= 2 THEN
        ADD "Multiple near misses" to warning
    ENDIF
    IF previous_incidents >= 2 THEN
        ADD "Multiple previous incidents" to warning
    ENDIF

    IF warning is empty THEN
        RETURN "No warning"
    ELSE
        REMOVE final comma and space from warning
        RETURN warning
    ENDIF
END safety_warning
```

show_worker_safety()

Purpose: Displays worker safety data, filters workers by risk level and shows workers requiring attention.

```
BEGIN show_worker_safety
    DISPLAY Worker Health and Safety heading
    INPUT selected risk filter

    IF selected risk filter = "All" THEN
        DISPLAY all workers
    ELSE
        FILTER workers WHERE Risk_Level = selected risk filter
        DISPLAY filtered workers
    ENDIF
```

```
    FILTER workers WHERE Safety_Warning is not equal to "No warning"
    DISPLAY Workers Requiring Attention
END show_worker_safety
```

PART C: SAFETY INCIDENT DATABASE

show_safety_incidents()

Purpose: Displays, searches, filters, adds and analyses safety incidents.

```
BEGIN show_safety_incidents
    DISPLAY Safety Incident Database heading
    DISPLAY total number of incidents

    INPUT incident ID to search
    IF incident ID is not blank THEN
        FIND incident WHERE Incident_ID = entered ID
        DISPLAY search result
    ENDIF

    INPUT selected severity
    IF selected severity = "All" THEN
        DISPLAY all incidents
    ELSE
        FILTER incidents WHERE Severity = selected severity
        DISPLAY filtered incidents
    ENDIF

    INPUT all fields in new incident form
    IF Save Incident button is clicked THEN
        IF Incident ID is blank THEN
            DISPLAY ID error
        ELSE IF Incident ID already exists THEN
            DISPLAY duplicate ID error
        ELSE IF Location is blank THEN
            DISPLAY location error
        ELSE IF Department is blank THEN
            DISPLAY department error
        ELSE IF Incident Type is blank THEN
            DISPLAY type error
        ELSE IF Cause is blank THEN
            DISPLAY cause error
        ELSE
            CREATE new incident record
            ADD record to incidents data
            SAVE incidents data to incidents.csv
            DISPLAY success message
            RERUN application
        ENDIF
    ENDIF

    COUNT incidents by Severity, Department, Shift and Incident_Type
    DISPLAY bar charts for Severity, Department, Shift and Incident Type
    COUNT incidents by Date and sort dates
    DISPLAY incident trend line chart
END show_safety_incidents
```

PART D: EQUIPMENT DATABASE

show_equipment_database()

Purpose: Displays equipment data and filters equipment according to condition status.

```
BEGIN show_equipment_database
    DISPLAY Equipment Monitoring heading
    DISPLAY complete equipment database
    INPUT selected condition

    IF selected condition = "All" THEN
        DISPLAY all equipment
    ELSE
        FILTER equipment WHERE Condition_Status = selected condition
        DISPLAY filtered equipment
    ENDIF
END show_equipment_database
```

PART E: EQUIPMENT CONDITION MONITORING

equipment_condition(temperature, vibration)

Purpose: Classifies equipment as Normal, Warning or Critical using temperature and vibration thresholds.

```
BEGIN equipment_condition
   INPUT temperature, vibration

   IF temperature > 100 OR vibration > 8 THEN
      RETURN "Critical"
   ELSE IF temperature >= 80 OR vibration >= 5 THEN
      RETURN "Warning"
   ELSE
      RETURN "Normal"
   ENDIF
END equipment_condition
```

show_equipment_condition()

Purpose: Displays a coloured condition message for each equipment item.

```
BEGIN show_equipment_condition
   DISPLAY Equipment Condition Status heading

   FOR EACH equipment item
      READ Equipment_ID and Condition_Status

      IF Condition_Status = "Critical" THEN
         DISPLAY red critical inspection message
      ELSE IF Condition_Status = "Warning" THEN
         DISPLAY yellow warning message
      ELSE
         DISPLAY green normal message
      ENDIF
   ENDFOR
END show_equipment_condition
```

PART F: EQUIPMENT MAINTENANCE MONITORING

Availability Calculation (pre-processing step)

Purpose: Calculates availability for every equipment item before maintenance functions are displayed.

```
BEGIN availability calculation
    FOR EACH equipment item
        READ Operating_Hours and Downtime
        Availability = Operating_Hours / (Operating_Hours + Downtime) * 100
        STORE Availability in equipment data
    ENDFOR
END availability calculation
```

maintenance_check(operating_hours, next_maintenance, current_status)

Purpose: Determines whether equipment is under maintenance, overdue, due soon or not due.

```
BEGIN maintenance_check
    INPUT operating_hours, next_maintenance, current_status

    IF current_status = "Under Maintenance" THEN
        RETURN "Under Maintenance"
    ELSE IF operating_hours > next_maintenance THEN
        RETURN "Overdue"
    ELSE IF operating_hours >= next_maintenance - 20 THEN
        RETURN "Due Soon"
    ELSE
        RETURN "Not Due"
    ENDIF
END maintenance_check
```

show_maintenance()

Purpose: Displays equipment requiring maintenance attention, low availability and maintenance alerts.

```
BEGIN show_maintenance
    DISPLAY Equipment Maintenance heading

    FILTER equipment WHERE Maintenance_Check is not equal to "Not Due"
    DISPLAY Maintenance Status table

    FILTER equipment WHERE Availability < 85
    DISPLAY Low Availability table

    FOR EACH equipment item
        IF Maintenance_Check = "Overdue" THEN
            DISPLAY maintenance overdue error
        ELSE IF Maintenance_Check = "Due Soon" THEN
```

```
        DISPLAY maintenance due soon warning
      ENDIF

      IF Maintenance_Status = "Under Maintenance" THEN
        DISPLAY under maintenance information
      ENDIF
      IF Downtime > 30 THEN
        DISPLAY excessive downtime warning
      ENDIF
      IF Previous_Alerts >= 3 THEN
        DISPLAY repeated alerts warning
      ENDIF
      IF Availability < 85 THEN
        DISPLAY low availability warning
      ENDIF
    ENDFOR
END show_maintenance
```

PART G: RISK ASSESSMENT

risk_assessment(likelihood, consequence)

Purpose: Calculates RiskScore and assigns Low, Medium, High or Critical risk.

```
BEGIN risk_assessment
    INPUT likelihood, consequence
    SET score = likelihood * consequence

    IF score <= 4 THEN
        SET level = "Low"
    ELSE IF score <= 9 THEN
        SET level = "Medium"
    ELSE IF score <= 16 THEN
        SET level = "High"
    ELSE
        SET level = "Critical"
    ENDIF

    RETURN score, level
END risk_assessment
```

show_risk_assessment()

Purpose: Accepts likelihood and consequence from the user, calculates the risk and displays the result.

```
BEGIN show_risk_assessment
    DISPLAY Risk Assessment heading
    INPUT likelihood from 1 to 5
    INPUT consequence from 1 to 5

    IF Calculate Risk button is clicked THEN
        score, level = risk_assessment(likelihood, consequence)
        DISPLAY Risk Score
        DISPLAY Risk Level

        IF level = "Critical" THEN
            DISPLAY critical risk error
        ELSE IF level = "High" THEN
            DISPLAY high risk warning
        ELSE
            DISPLAY risk assessment completed message
        ENDIF
    ENDIF
END show_risk_assessment
```

PART H: ALERTS AND NOTIFICATIONS

equipment_alert(temperature, vibration, brakes, tyres, engine, maintenance)

Purpose: Builds an equipment alert message using condition, component and maintenance values.

```
BEGIN equipment_alert
    INPUT temperature, vibration, brakes, tyres, engine, maintenance
    SET alert = empty text

    IF temperature > 100 THEN
        ADD "Critical temperature" to alert
    ELSE IF temperature >= 80 THEN
        ADD "High temperature" to alert
    ENDIF

    IF vibration > 8 THEN
        ADD "Critical vibration" to alert
    ELSE IF vibration >= 5 THEN
        ADD "High vibration" to alert
    ENDIF

    IF brakes is not equal to "Good" THEN
        ADD "Check brakes" to alert
    ENDIF
    IF tyres is not equal to "Good" THEN
        ADD "Check tyres" to alert
    ENDIF
    IF engine is not equal to "Good" THEN
        ADD "Check engine" to alert
    ENDIF
    IF maintenance = "Overdue" THEN
        ADD "Maintenance overdue" to alert
    ENDIF

    IF alert is empty THEN
        RETURN "No alert"
    ELSE
        REMOVE final comma and space
        RETURN alert
    ENDIF
END equipment_alert
```

show_alerts(role)

Purpose: Displays worker and/or equipment alerts according to the logged-in user role.

```
BEGIN show_alerts
    INPUT role
    DISPLAY Alerts and Notifications heading
```

```
    IF role is not equal to "Maintenance Engineer" THEN
        FILTER workers WHERE Safety_Warning is not equal to "No warning"
        DISPLAY Worker Safety Alerts table

        FOR EACH worker
            IF Risk_Level = "Critical" THEN
                DISPLAY critical worker safety alert
            ELSE IF Risk_Level = "High" THEN
                DISPLAY high worker safety warning
            ENDIF
        ENDFOR
    ENDIF

    IF role is not equal to "Safety Officer" THEN
        FILTER equipment WHERE Equipment_Alert is not equal to "No alert"
        DISPLAY Equipment Alerts table

        FOR EACH equipment item
            IF Condition_Status = "Critical" THEN
                DISPLAY critical equipment alert
            ELSE IF Condition_Status = "Warning" THEN
                DISPLAY equipment warning
            ENDIF
        ENDFOR
    ENDIF
END show_alerts
```

PART I: MINING MONITORING DASHBOARD

show_dashboard()

Purpose: Calculates and displays the eleven dashboard KPIs required by the application.

```
BEGIN show_dashboard
   SET total_workers = number of workers
   SET total_incidents = number of incidents
   SET total_near_misses = sum of Near_Misses
   SET total_equipment = number of equipment items
   SET ppe_yes = 0
   SET high_risk_observations = 0
   FOR EACH worker
      IF PPE_Compliance = "Yes" THEN
         ppe_yes = ppe_yes + 1
      ENDIF
      IF Risk_Level = "High" OR Risk_Level = "Critical" THEN
         high_risk_observations = high_risk_observations + 1
      ENDIF
   ENDFOR
   IF number of workers > 0 THEN
      ppe_percentage = ppe_yes / number of workers * 100
   ELSE
      ppe_percentage = 0
   ENDIF
   SET available = 0
   SET under_maintenance = 0
   SET equipment_alert_count = 0
   SET critical_alerts = 0
   FOR EACH equipment item
      IF Maintenance_Status = "Available" THEN
         available = available + 1
      ENDIF
      IF Maintenance_Status = "Under Maintenance" THEN
         under_maintenance = under_maintenance + 1
      ENDIF
      IF Equipment_Alert is not equal to "No alert" THEN
         equipment_alert_count = equipment_alert_count + 1
      ENDIF
      IF Condition_Status = "Critical" THEN
         critical_alerts = critical_alerts + 1
      ENDIF
   ENDFOR
   IF number of equipment items > 0 THEN
      average_availability = sum of Availability / number of equipment items
   ELSE
      average_availability = 0
   ENDIF
   DISPLAY 11 KPI cards; END
```

PART J: INTERACTIVE DATA ANALYSIS

most_common(column)

Purpose: Finds the most frequently occurring value in a selected data column.

```
BEGIN most_common
    INPUT column
    CREATE empty counts dictionary

    FOR EACH item in column
        IF item already exists in counts THEN
            counts[item] = counts[item] + 1
        ELSE
            counts[item] = 1
        ENDIF
    ENDFOR

    SET answer = empty text
    SET highest = 0

    FOR EACH item in counts
        IF counts[item] > highest THEN
            highest = counts[item]
            answer = item
        ENDIF
    ENDFOR

    RETURN answer
END most_common
```

highest_downtime()

Purpose: Finds the Equipment_ID with the greatest downtime.

```
BEGIN highest_downtime
    SET equipment_id = first Equipment_ID
    SET highest = first Downtime value

    FOR EACH equipment item
        IF current Downtime > highest THEN
            highest = current Downtime
            equipment_id = current Equipment_ID
        ENDIF
    ENDFOR

    RETURN equipment_id
END highest_downtime
```

lowest_availability()

Purpose: Finds the Equipment_ID with the lowest availability.

```
BEGIN lowest_availability
    SET equipment_id = first Equipment_ID
    SET lowest = first Availability value

    FOR EACH equipment item
        IF current Availability < lowest THEN
            lowest = current Availability
            equipment_id = current Equipment_ID
        ENDIF
    ENDFOR

    RETURN equipment_id
END lowest_availability
```

serious_incident_count()

Purpose: Counts incidents classified as High or Critical.

```
BEGIN serious_incident_count
    SET count = 0

    FOR EACH incident
        IF Severity = "High" THEN
            count = count + 1
        ELSE IF Severity = "Critical" THEN
            count = count + 1
        ENDIF
    ENDFOR

    RETURN count
END serious_incident_count
```

ppe_compliance_percentage()

Purpose: Calculates the percentage of workers who comply with PPE requirements.

```
BEGIN ppe_compliance_percentage
    SET compliant = 0
    FOR EACH worker
        IF PPE_Compliance = "Yes" THEN
            compliant = compliant + 1
        ENDIF
    ENDFOR
    IF number of workers > 0 THEN
        percentage = compliant / number of workers * 100
    ELSE
```

```
    percentage = 0
  ENDIF
  RETURN percentage rounded to 1 decimal place
END ppe_compliance_percentage
```

highest_temperature()

Purpose: Finds the Equipment_ID with the highest recorded temperature.

```
BEGIN highest_temperature
  SET equipment_id = first Equipment_ID
  SET highest = first Temperature value

  FOR EACH equipment item
    IF current Temperature > highest THEN
      highest = current Temperature
      equipment_id = current Equipment_ID
    ENDIF
  ENDFOR

  RETURN equipment_id
END highest_temperature
```

highest_vibration()

Purpose: Finds the Equipment_ID with the highest recorded vibration.

```
BEGIN highest_vibration
  SET equipment_id = first Equipment_ID
  SET highest = first Vibration value

  FOR EACH equipment item
    IF current Vibration > highest THEN
      highest = current Vibration
      equipment_id = current Equipment_ID
    ENDIF
  ENDFOR

  RETURN equipment_id
END highest_vibration
```

answer_question(question)

Purpose: Matches the selected analysis question to the correct helper function and returns the answer.

```
BEGIN answer_question
  INPUT question
```

```
    IF question asks for department with most incidents THEN
        RETURN most_common(Department)
    ELSE IF question asks for shift with most incidents THEN
        RETURN most_common(Shift)
    ELSE IF question asks for most common incident type THEN
        RETURN most_common(Incident_Type)
    ELSE IF question asks for most common cause THEN
        RETURN most_common(Cause)
    ELSE IF question asks for highest downtime THEN
        RETURN highest_downtime()
    ELSE IF question asks for lowest availability THEN
        RETURN lowest_availability()
    ELSE IF question asks for High/Critical incident count THEN
        RETURN serious_incident_count()
    ELSE IF question asks for PPE compliance percentage THEN
        RETURN ppe_compliance_percentage() with % sign
    ELSE IF question asks for highest temperature THEN
        RETURN highest_temperature()
    ELSE IF question asks for highest vibration THEN
        RETURN highest_vibration()
    ELSE
        RETURN "Question not available"
    ENDIF
END answer_question
```

show_data_analysis(role)

Purpose: Controls which analysis questions and charts are available to each role.

```
BEGIN show_data_analysis
    INPUT role
    CREATE safety_questions list
    CREATE equipment_questions list

    IF role = "Safety Officer" THEN
        allowed_questions = safety_questions
    ELSE IF role = "Maintenance Engineer" THEN
        allowed_questions = equipment_questions
    ELSE
        allowed_questions = all questions
    ENDIF

    INPUT selected question from allowed_questions

    IF Answer Question button is clicked THEN
        answer = answer_question(selected question)
        DISPLAY answer
    ENDIF

    IF role is not equal to "Maintenance Engineer" THEN
        DISPLAY incidents by Department chart
```

```
    DISPLAY incidents by Severity chart
  ENDIF

  IF role is not equal to "Safety Officer" THEN
    DISPLAY Equipment Condition chart
  ENDIF
END show_data_analysis
```

PART APPENDIX: REPORT FUNCTION OUTSIDE PARTS A–J

show_reports(role)

Purpose: Generates downloadable worker, incident and equipment reports according to role permissions.

```
BEGIN show_reports
   INPUT role
   DISPLAY Reports heading

   IF role is not equal to "Maintenance Engineer" THEN
      DISPLAY Worker Safety Report
      CONVERT workers data to CSV
      DISPLAY Download Worker Report button

      DISPLAY Safety Incident Report
      CONVERT incidents data to CSV
      DISPLAY Download Incident Report button
   ENDIF

   IF role is not equal to "Safety Officer" THEN
      DISPLAY Equipment Report
      CONVERT equipment data to CSV
      DISPLAY Download Equipment Report button
   ENDIF
END show_reports
```